

SOC Engineering

Vizsgafelkészítő útmutató

Logbekötés · Parsing · Riasztás · Dashboard · Log- és forgalomelemzés

Filebeat → Logstash → OpenSearch

Tartalom

1. Áttekintés – a teljes log pipeline

A feladatsor egy klasszikus log pipeline felépítésére és használatára épül. A lánc neve onnan ered, hogy az adat több, egymásra épülő állomáson halad át, és minden komponens egyetlen dolgot csinál jól, majd átadja a következőnek. Mielőtt a részfeladatokba kezdenél, érdemes az egész láncot átlátni, mert minden további lépés (parsing, riasztás, dashboard, elemzés) erre épül.

Az adatfolyam a következő: a log fájlok (/var/log/exam) tartalmát a Filebeat begyűjti és továbbítja a Logstash-nek; a Logstash strukturálja (parse) az adatot, majd elküldi az OpenSearch-be; az OpenSearch tárolja és indexeli; végül az OpenSearch Dashboards felületén keresel, vizualizálsz és riasztasz.

A komponensek szerepe

- Filebeat – könnyűsúlyú log-szállító (shipper). Ott fut, ahol a log keletkezik, figyeli a megadott fájlokat, és az új sorokat továbbítja. Szándékosan „buta”: nem értelmez, csak begyűjt és továbbít, ezért alig fogyaszt erőforrást, és sok végpontra kitehető.
- Logstash – feldolgozó motor három szakasszal: input (honnan jön), filter (mit csinálunk vele – itt strukturál a grok), output (hová megy tovább). Központilag fut, ide érkezik az összes forrás.
- OpenSearch – tároló- és keresőmotor (az Elasticsearch nyílt forrású továbbfejlesztése). Indexeli az adatot, hogy nagy mennyiségben is gyorsan kereshető legyen.
- OpenSearch Dashboards – a webes felület (a Kibana megfelelője): keresés, vizualizáció és riasztás.

Egy fontos fogalmi pont a vizsgára: miért megyünk Logstash-en át, és nem közvetlenül Filebeat → OpenSearch? Mert a Logstash-ben bontjuk a logot mezőkre (grok). Ezért a Filebeat kimenetét nem az elasticsearch-re, hanem a logstash-re állítjuk.

2. Logbekötés: Filebeat → Logstash → OpenSearch

Cél: az adat eljusson a fájloktól az OpenSearch-ig. A parse-olás még nem itt történik – egyelőre az is elég, ha a nyers log megjelenik a felületen.

Lépésről lépésre

2.1 SSH és a logok megkeresése

Bejelentkezel a VM-re SSH-val, majd megnézed, milyen logok vannak és mi a nevük:

```
ls -l /var/log/exam/  
head -n 5 /var/log/exam/vpn.log
```

Az egy-két minta sor megnézése azért fontos, mert a következő feladatban a log formátumából írod meg a grok mintát.

2.2 A Filebeat konfiguráció megismerése

A konfiguráció a `/etc/filebeat/` könyvtárban van. A `filebeat.yml` az éles konfiguráció, amit szerkesztünk; a `filebeat.reference.yml` egy teljes referencia minden beállítás kifejtett kommentjével (ide nézel, ha elakadsz); a `modules.d/` mappa előre elkészített modulokat tartalmaz ismert szoftverekhez (most nem ezt használjuk).

2.3 Az input beállítása

Nyisd meg szerkesztésre, és állítsd be a `filebeat.inputs:` szakaszt (a YAML behúzás kritikus – szóközzel, nem tabbal):

```
filebeat.inputs:
- type: log
  enabled: true
  paths:
  - /var/log/exam/vpn.log
  fields:
    log_type: vpn
  fields_under_root: true
```

- `enabled: true` – ez aktiválja az inputot. Ha kihagyod, semmi nem olvas be – gyakori kezdő hiba.
- `fields: log_type: vpn` – saját címke minden eseményhez. Ebből tudja majd a Logstash, melyik grok filtert alkalmazza, és e szerint külön indexbe is rakhatod a típusokat. Több forrásnál mindegyikhez más `log_type`-ot adsz.
- `fields_under_root: true` – e nélkül a mező egy `fields.log_type` aldobozba kerülne; ezzel a dokumentum gyökerébe (`log_type`), így egyszerűbb rá hivatkozni és szűrni.

Megjegyzés: újabb Filebeat-verziókban a `type: log` helyett `filestream` az ajánlott típus – a logika ugyanaz.

2.4 Az output átállítása Logstash-re

Az Elasticsearch kimenetet kikommentezzük (nem oda küldünk most), és a Logstash kimenetet kapcsoljuk be:

```
# output.elasticsearch:
#   hosts: ["localhost:9200"]

output.logstash:
  hosts: ["localhost:5044"]
```

Az 5044 a Logstash beats-inputjának alapértelmezett portja – ezen kommunikál a Filebeat és a Logstash.

2.5 Ellenőrzés és újraindítás

Indítás előtt érdemes ellenőrizni a konfigurációt:

```
sudo filebeat test config
sudo filebeat test output
```

Ha mindkettő OK, jöhet az újraindítás és a státusz (a szolgáltatás neve `filebeat`, a `systemctl` magától érti a `.service` végződést):

```
sudo systemctl restart filebeat
```

```
sudo systemctl status filebeat
```

2.6 A Logstash fogadóoldala

A Logstash a /opt-ban van, a configja a /opt/logstash/config/conf.d/pipeline.conf. A filter részt a következő feladatban írod meg, de az 1. feladat működéséhez az input és output szakasznak állnia kell:

```
input {
  beats {
    port => 5044
  }
}

filter {
  # Ide jön a grok a következő feladatban – egyelőre üres
}

output {
  opensearch {
    hosts => ["https://localhost:9200"]
    index => "vpn-%{+YYYY.MM.dd}"
    user => "admin"
    password => "<a labor jelszava>"
    ssl_certificate_verification => false
  }
}

sudo systemctl restart logstash
sudo systemctl status logstash
```

2.7 Ellenőrzés az OpenSearch felületén

A Dashboardsban hozz létre index pattern-t: Dashboard Management → Index Patterns → Create, a minta vpn-*, időmezőnek a @timestamp. A Discover nézetben ezután látnod kell a beérkező sorokat (még struktúrálatlanul, egyetlen message mezőben).

3. Strukturálás: grok parsing a Logstash-ben

Amíg minden a message mezőben van, nem tudsz értelmesen keresni vagy riasztani. A grok dolga, hogy a nyers sorból kereshető, aggregálható mezőket csináljon (pl. src_ip, username, result, geoip.country_name).

A grok alapjai

A grok mintaillesztő nyelv, regex tetejére építve, sok előre megírt mintával. Az alapszintaxis:

```
%{MINTA:mezőnév}
```

Például a %{IP:src_ip} annyit tesz: „itt egy IP-cím következik, tedd a src_ip mezőbe”. A fix szövegrészeket (user=, : stb.) szó szerint kell a minták közé írni – ezek horgonyok.

A leggyakoribb beépített minták:

Minta	Mit illeszt
IP / IPORHOST	IP-cím / IP vagy hostnév

NUMBER / INT	szám / egész szám
WORD	szóköz nélküli szó (betűk, számok)
NOTSPACE	bármilyen, ami nem szóköz
USERNAME	felhasználónév-szerű karakterek
TIMESTAMP_ISO8601	ISO formátumú időbélyeg (pl. 2026-06-03T14:22:01Z)
LOGLEVEL	INFO / WARN / ERROR stb.
DATA / GREEDYDATA	„bármilyen” – óvatosan, mohó, a sor végére jó

Hogyan írsz grok mintát?

A trükk: ne fejből, hanem a tényleges sorból építsd fel, és tesztelj.

1. Nézd meg a pontos formátumot egy mintán. Tegyük fel, egy sor így néz ki:

```
2026-06-03T14:22:01Z gateway1 vpn: user=jdoe src=203.0.113.45 action=connect result=success
```

2. Darabold fel: időbélyeg → `TIMESTAMP_ISO8601`, `gateway1` → `WORD`, `vpn` → `WORD`, `jdoe` → `USERNAME`, `IP` → `IP`, `connect/success` → `WORD`.

3. Rakd össze a mintát:

```
%{TIMESTAMP_ISO8601:timestamp} %{WORD:host} %{WORD:service}:
user=%{USERNAME:username} src=%{IP:src_ip}
action=%{WORD:action} result=%{WORD:result}
```

(A fenti minta egy sorba kerül a configban; itt csak az olvashatóság miatt tördeltük.)

4. Teszteld, mielőtt élesítenéd. Az OpenSearch Dashboardsban a Dev Tools → Grok Debugger azonnal megmutatja, milyen mezőkre bontott. Fokozatosan bővítsd a mintát.

A teljes filter blokk

A vizsgán több forrásod lesz, különböző formátummal – itt jön elő a `log_type` mező haszná: ennek alapján feltételelesen futtatod a megfelelő grokot.

```
filter {
  if [log_type] == "vpn" {
    grok {
      match => {
        "message" => "%{TIMESTAMP_ISO8601:timestamp} %{WORD:host} %{WORD:service}: user=%{USERNAME:username} src=%{IP:src_ip} action=%{WORD:action} result=%{WORD:result}"
      }
    }
    date {
      match => [ "timestamp", "ISO8601" ]
      target => "@timestamp"
    }
    geoip {
      source => "src_ip"
      target => "geoip"
    }
    mutate {
      remove_field => [ "message", "timestamp" ]
    }
  }
}
```

```

}
}
# else if [log_type] == "gateway" { grok { ... } }
}

```

- `date` – a kinyert időbélyeget beállítja az esemény hivatalos idejévé (`@timestamp`). E nélkül az `@timestamp` a feldolgozás ideje lenne, nem a tényleges eseményé.
- `geoip` – a `src_ip` alapján földrajzi adatot (ország, koordináták) tesz hozzá. Ez közvetlenül a riasztási feladatot készíti elő: az „EU-n kívüli ország” feltétel a `geoip.country_iso_code` mezőre épül.
- `mutate` – feleslegessé vált mezőket dob el; típust is konvertálhat (`convert`). Tipp: amíg a `grok` nem tökéletes, hagyd benn a `message-t`, hogy lásd az eredetit.

Élesítés után indítsd újra a Logstash-t, és nézd a logját hiba esetén:

```

sudo systemctl restart logstash
sudo journalctl -u logstash -f

```

A Discoverben az új sorokon megjelennek a mezők; ha nem, frissítsd az index pattern mezőlistáját. Sikertelen parse esetén a `_grokparsefailure` címke jelzi, hogy a minta nem illeszkedik – vissza a Grok Debuggerhez.

4. Riasztás Teams webhookkal

Válaszd külön a két oldalt: az OpenSearch oldalt (mit figyelünk, mikor riasztunk) és a Teams oldalt (hová érkezik az üzenet).

Az OpenSearch Alerting felépítése

Egy Monitor tartalmaz egy vagy több Triggert, egy Trigger pedig egy vagy több Actiont. Az Action egy Channelen (custom webhook) keresztül üzen a Teamsnek.

- **Monitor** – melyik indexet, milyen lekérdezéssel, milyen ütemezéssel figyeljen. Típusai közül kezdésnek a `per query` a legkönnyebb.
- **Trigger** – feltétel a monitor eredményére (pl. „van legalább egy találat”), `Painless` kifejezéssel, a `ctx` objektumon át.
- **Action** – ha a Trigger tüzel, üzenetet küld. Itt írod meg az üzenetet `Mustache` sablonnal (`{{ctx....}}`).
- **Channel** – a Notifications pluginban; Teamshez a Custom webhook típus a Teams URL-jével.

Fontos: a Teams webhook 2026-ban változott

A klasszikus Teams „incoming webhook” (régi Office 365 / M365 connector) kifutóban van: a connector-alapú webhooksokat 2026 májusáig át kellett migrálni Workflows-ra, és a határidő után a régi connector-végpontok leállnak. A mai dátummal ezek jó eséllyel már nem működnek.

A jelenlegi megoldás a Power Automate Workflows alapú webhook: a Teams-csatorna „...” menüjéből Workflows → „Send webhook alerts to a channel” sablon, és a kapott URL-t használd. Formátumbeli következmény: a régi „MessageCard” JSON helyett Adaptive Card

payloadot küldesz. A vizsgán a labor előre beállított Teams-végpontját és az általa elvárt formátumot használd – alább mindkét formátum szerepel.

Lépésről lépésre

5. Szerezd meg a Teams webhook URL-t (a labor adja, vagy a Workflows sablonnal hozod létre).
6. Hozz létre Notification Channelt: Notifications → Channels → Create channel → Custom webhook. Illeszd be az URL-t, és állíts be fejléctet: Content-Type: application/json. A „Send test message” gombbal azonnal tesztelhetsz.
7. Hozd létre a Monitort: Alerting → Monitors → Create monitor. Add meg a nevet, típust (Per query), az indexet (vpn-*), az ütemezést, és a lekérdezést. A „sikeres belépés ÉS nem EU-s ország” szabály query DSL-ben:

```
{
  "query": {
    "bool": {
      "must": [
        { "match": { "result": "success" } }
      ],
      "must_not": [
        { "terms": { "geoip.country_iso_code":
          ["AT","BE","BG","HR","CY","CZ","DK","EE","FI","FR","DE",
            "GR","HU","IE","IT","LV","LT","LU","MT","NL","PL","PT",
            "RO","SK","SI","ES","SE"] } }
      ]
    }
  }
}
```

A logika: vedd a sikeres belépéseket (must), és zárd ki az EU-tagállamokból érkezőket (must_not). Ami marad: sikeres belépés EU-n kívülről. Érdemes időablakot (pl. utolsó 1 perc) is tenni rá.

8. Definiáld a Triggert. A legegyszerűbb feltétel Painlessben:

```
ctx.results[0].hits.total.value > 0
```

9. Add hozzá az Actiont, válaszd ki a Channelt, és írd meg az üzenetet. Workflows / Adaptive Card formátum:

```
{
  "type": "message",
  "attachments": [{
    "contentType": "application/vnd.microsoft.card.adaptive",
    "content": {
      "type": "AdaptiveCard",
      "version": "1.4",
      "body": [
        { "type": "TextBlock", "size": "Large", "weight": "Bolder",
          "text": "OpenSearch riasztás: {{ctx.monitor.name}}" },
        { "type": "TextBlock", "wrap": true,
          "text": "Sikeres bejelentkezés EU-n kívülről. Találatok: {{ctx.results.0.hits.total.value}}" }
      ]
    }
  ]
}
```

```
}  
}
```

Ha a labor a régi connectort használja, az egyszerűbb MessageCard formátum kell:

```
{  
  "@type": "MessageCard",  
  "@context": "http://schema.org/extensions",  
  "summary": "OpenSearch riasztás",  
  "themeColor": "D7263D",  
  "title": "Riasztás: {{ctx.monitor.name}}",  
  "text": "Sikeres bejelentkezés EU-n kívülről. Találatok: {{ctx.results.0.hits.total.value}}"  
}
```

10. Mentés és tesztelés. Az Action melletti „Send test message” teszteli a teljes láncot. Gyakori hibák: hiányzó Content-Type fejléc, rossz payload-formátum (MessageCard vs Adaptive Card), lejárt URL. Végül teszteld éles eseménnyel is.

5. Dashboardok

A dashboard több kis grafikon (visualization) egy felületen, közös időablakkal és szűrőkkel. A parsing előállította mezőkre épül minden.

Három szint

- Index pattern – melyik indexeket olvassa (vpn-*) és melyik az idő mező (@timestamp). E nélkül semmi nem működik.
- Visualization – egyetlen grafikon (oszlop, kör, táblázat, térkép, számláló), egy lekérdezés eredménye.
- Dashboard – több elmentett visualization egy vásznon, közös időválasztóval és szűrőkkel.

Az aggregáció modellje: bucket + metric

Minden grafikon két kérdésre válaszol. A metric a „mit mérünk” (Count, Unique Count, Average, Sum stb.). A bucket a „hogyan bontjuk fel” (Terms – top értékek, Date Histogram – idő szerinti vödrök, Range, Geo). Mentális kép: a bucket adja a tengelyt/szeleteket, a metric pedig azt, amit mérsz rajta.

Lépések

11. Discoverben nézd át a mezőket; csak létező, jól indexelt mezőre tudsz aggregálni.
12. Készíts egyesével vizualizációkat (Visualize → Create visualization):
 - Metric – összes belépés (Count); egyedi felhasználók (Unique Count a username-en).
 - Vertical Bar – események időben: metric Count, bucket Date Histogram a @timestamp-en.
 - Horizontal Bar – forrásországok: bucket Terms a geoip.country_name-en, méret 10.
 - Pie – sikeres vs. sikertelen: bucket Terms a result mezőn.
 - Data Table – top felhasználók: bucket Terms a username-en, metric Count.

- Térkép – a geoip.location mezőből, vizuálisan az EU-n kívüli belépésekhez.
13. Rakd össze: Dashboard → Create → Add, válaszd ki az elmentett vizualizációkat, méretezd és rendezd el.
 14. Állítsd be az időablakot, adj globális szűrőt (pl. result: success), tehetsz Control elemeket is, majd mentsd.

Tippek

Ha a Date Histogram üres, gyakran rossz az időmező vagy túl szűk az időablak. Ha egy mezőre nem megy a Terms aggregáció, az gyakran text típus – használd a .keyword változatot (pl. username.keyword). Építsd a dashboardot SOC-logikával: összkép fent, bontás középen, részletek (táblázat) lent.

6. Logelemzés

Itt nem építesz, hanem a bekötött adatból válaszolsz kérdésekre, főleg a Discover nézetben, a kiparse-olt mezőkre építve. A lekérdezőnyelv a DQL (Dashboards Query Language): mező: érték, logikai operátorokkal.

```
result: failure
result: success and not geoip.country_iso_code: "HU"
src_ip: "203.0.113.45"
username: admin*
bytes >= 1000
_exists_: error_code
```

Munkamenet: szűrj, majd pivotálj

Példa – „Történt-e brute-force, melyik IP-ről, sikerült-e?” Először szűrj result: failure-re, és a bal oldali mezőlistában a src_ip-re kattintva nézd a top értékeket. Ha egy IP kiugrik (pl. 312 sikertelen kísérlettel), pivotálj rá: szűrj src_ip: "..."-re, vedd le a failure szűrőt, és rendezd időrendbe. A brute-force mintája: sok failure rövid időablakban, majd egyetlen success. Ha a sorozat végén ott a result: success, a támadás sikeres volt, és kiolvasod, melyik username-mel. Ezt egyetlen message mezőből nem tudnád – ezért érte meg a parse-olás.

Mire figyelj

- Időablak – ha üresnek tűnik a lekérdezés, szinte mindig túl szűk a felső időválasztó. Állítsd tágra először.
- text vs. keyword – pontos egyezésnél és aggregációnál gyakran a .keyword változat kell.
- Időzóna – a logok jellemzően UTC-ben vannak, a Dashboards helyi időre konvertál; ez „eltolhatja” az eseményeket.
- Ne egyetlen eseményből következtess – mintázatokat keress (számok, sorozatok, ismétlődés).

7. Forgalomelemzés (pcap)

A pcap a hálózati forgalom nyers rögzítése (minden csomag minden rétegével). Fő eszköz a Wireshark (grafikus), CLI-ből a tshark és a capinfos. A display filter szintaxisa pont-alapú: `ip.addr == 203.0.113.45, tcp.port == 80, http, dns`.

Tölcsér-munkamenet: áttekintés → szűkítés → tartalom

Áttekintéshez a Statistics menü: Protocol Hierarchy (milyen protokollok vannak), Conversations (mely IP-párok beszéltek és mennyit), Endpoints (résztevő gépek). Gyors összefoglaló CLI-ből:

```
capinfos fajl.pcap
```

Példa: elküldött felhasználónév és jelszó

A Protocol Hierarchy-ban látszik a HTTP forgalom (titkosítatlan). Szűkíts a POST kérésekre, mert a bejelentkezés tipikusan POST:

```
http.request.method == "POST"
```

Találsz egy POST-ot a /login útvonalra. Jobb gomb → Follow → HTTP Stream: összerakja a teljes kérés-választ, és a kérés törzsében ott az űrlap, pl. `username=admin&password=Passw0rd123`. tshark-kal egy sorban:

```
tshark -r fajl.pcap -Y 'http.request.method=="POST"' -T fields -e http.file_data
```

További tipikus kérdések

- Lekérdezett domain – szűrj dns-re, a DNS query nevekben látod a domaineket (gyanús esetben pl. C2-szerver).
- Portszenkálás – jellegzetes mintája sok SYN egy forrásból sok portra, válasz nélkül:

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

- Letöltött fájl – File → Export Objects → HTTP menüből kimentheted.

Mire figyelj

- Follow Stream – a legfontosabb eszköz; titkosítatlan protokolloknál (HTTP, FTP, Telnet) itt látszanak a jelszavak, parancsok, fájlnevek.
- Titkosítás – HTTPS/TLS esetén kulcs nélkül nem olvasod a tartalmat; a metaadatokra (IP, SNI-domain, adatmennyiség) támaszkodj.
- Display filter vs. capture filter – a vizsgán szinte mindig display filtert írsz (már rögzített fájlra szűr).

8. Összegzés – a közös gondolkodásmód

A teljes feladatsor egy logikai ívet követ. Először felépíted az adatutatót (Filebeat → Logstash → OpenSearch), majd a Logstash-ben grokkal mezőkre bontod a logot. A strukturált adatra építed a riasztást (Monitor → Trigger → Action → Channel → Teams) és a dashboardot (index pattern → visualization → dashboard, bucket + metric logikával). Végül a kész adatból és a rögzített forgalomból nyomozói munkával válaszolsz kérdésekre.

Az elemzési feladatok közös fonala ugyanaz, mint egy valódi SOC-műszakban L1-ként: átfogó kép → szűkítés a gyanúsra → a tartalom vagy mintázat elolvasása → következtetés. A logelemzésnél a parse-olt mezők és a DQL a szerszámod, a forgalomelemzésnél a Wireshark statisztikái és a display filterek – de a gondolkodás iránya azonos.